

Skill 3 · Referência — Exemplos e goldens

Versão: v1.11 · Referência da Skill 3 · Progressive disclosure

Skill interna — carregar **somente** quando o núcleo (`skill-03-base-conversao-lsp-java.md`) indicar esta âncora. Aplique as regras globais do Router.

Exemplos sanitizados (use só o análogo)

HorSis + setHorSit

```
Se (HorSis > 870) Inicio
  HorSit = 120;   @ situação 100 @
Fim;
```

```
java.time.LocalDateTime agora = java.time.LocalDateTime.now();
int minutos = agora.getHour() * 60 + agora.getMinute();
if (minutos > 870) {
    contextoApuracao.setHorSit(100, 120);
}
```

HorSit — errado vs certo

```
// ERRADO
contexto.getSituacao(xNorAnt).setMinutos(0);
// CERTO
contexto.zeraHorasSituacao(xNorAnt);
int v = contexto.getHorSit(xExDDsr); // minutos int
contexto.setHorSit(xExDDsr, v + vNorDes);
```

Marcações FLeMar/FPxMar → List

```
for (MarcacaoRegra m : contextoApuracao.getMarcacoesRealizadas(false)) {
    LocalDate d = m.getData().toLocalDate();
    if (d.isAfter(contextoApuracao.getData())) { /* ... */ }
}
```

End → retorno (RetBHRDat)

```
// ordem Java ≠ LSP – confirmar no índice
int saldo = contextoApuracao.getSaldoBanco(codbhr, numemp, tipcol, numcad,
    new LocalDate(2014, 9, 24));
```

Múltiplos End → objeto

```
private ResultadoDiurnoNoturno calculaDiurnoNoturno(...) {
    return new ResultadoDiurnoNoturno(minutosDiurnos, minutosNoturnos);
}
```

Troca de situação

```
private int trocarSituacao(ContextoApuracao ctx, int origem, int destino, int minutosPendentes) {
    int minutosOrigem = ctx.getHorSit(origem);
    if (minutosOrigem >= minutosPendentes && minutosOrigem > 0 && minutosPendentes > 0) {
        ctx.setHorSit(destino, minutosPendentes);
        ctx.setHorSit(origem, minutosOrigem - minutosPendentes);
        return 0;
    }
    if (minutosOrigem < minutosPendentes && minutosOrigem > 0 && minutosPendentes > 0) {
        ctx.setHorSit(destino, minutosOrigem);
        ctx.setHorSit(origem, 0);
        return minutosPendentes - minutosOrigem;
    }
    return minutosPendentes;
}
```

CalculaQtdMinutos (virada de dia)

```
private int calculaQtdMinutos(LocalDate d1, int horIni, LocalDate d2, int horFim) {
    if (d1.isBefore(d2)) horFim = horFim + 1440;
    return horFim - horIni;
}
```

TipCon via ContextSession

```
int nTipCon = 0;
try {
    ResultSet rs = ContextSession.getSession().executeQuery(
        "SELECT TIPCON FROM R034FUN WHERE NUMEMP = ? AND TIPCOL = ? AND NUMCAD = ?",
        numEmp, tipCol, numCad);
    if (rs.next()) nTipCon = rs.getInt(0);
} catch (Exception ex) {
    ctx.mensagemLog("Erro TipCon: " + ex.getMessage());
}
```

Interjornada (MarcacaoAnterior)

```

if (ctx.getMarcacaoAnterior() != null) {
    int n = ctx.getMarcacaoAnterior().diferencaMinutos(datMar, horaMar);
    if (n < 0) n = -n;
    // preferir getHorasInterjornada* quando disponível no contexto
}

```

EntitySession / ICursor (USU_* sem API)

```

ICursor<IEntidadeCustom> cursor = getContainer().getEntitySession()
    .newCursor(IEntidadeCustom.class);
try {
    cursor.addFilter("USU_Campo = :v", new MappedParamProvider("v", valor));
    cursor.open();
    if (cursor.first()) { /* read */ }
} finally {
    cursor.close();
}

```

Justificar ausência de API semântica; emitir interface+ .sc só para USU_* .

DiferencaMarcacaoDiurnoNoturno (sanitizado)

Preferir getHorasSeparadas / APIs oficiais quando cobrirem o caso. Se a regra LSP calcular manualmente diurno/noturno na janela 22h–05h (1320 / 300):

```

/** Retorna [diurno, noturno] em minutos. Janela noturna padrão: 1320→1440 e 0→300.
 */
private int[] diferencaMarcacaoDiurnoNoturno(int nMar1, LocalDate d1, int nMar2,
LocalDate d2,
                                           int iniNot, int fimNot) {
    int total = calculaQtdMinutos(d1, nMar1, d2, nMar2);
    if (total < 0) total = -total;
    int noturno = 0;
    int cursor = nMar1;
    LocalDate dia = d1;
    int restante = total;
    while (restante > 0) {
        int limiteNot = (cursor >= iniNot) ? 1440 : (cursor < fimNot ? fimNot :
iniNot);
        int trecho;
        if (cursor >= iniNot || cursor < fimNot) {
            trecho = Math.min(restante, (cursor >= iniNot ? 1440 - cursor : fimNot -
cursor));
            if (trecho <= 0) trecho = Math.min(restante, 1);
            noturno += trecho;
        } else {
            trecho = Math.min(restante, iniNot - cursor);
        }
        cursor += trecho;
        restante -= trecho;
    }
}

```

```

        if (cursor >= 1440) {
            cursor -= 1440;
            dia = dia.plusDays(1);
        }
    }
    return new int[]{total - noturno, noturno};
}

```

Proibido publicar `return new int[]{0,0}` ou `// preencher` — isso é `CHK-STUB FAIL`.

VerificaInterjornada (MarcacaoAnterior)

```

private void verificaInterjornada(ContextoApuracao ctx, LocalDate datMar, int
horaMar, int sitInt) {
    if (ctx.getMarcacaoAnterior() == null) return;
    // Preferir getHorasInterjornadaRealizada/Prevista quando existirem no contexto
    int n = ctx.getMarcacaoAnterior().diferencaMinutos(datMar, horaMar);
    if (n < 0) n = -n;
    if (n > 0 && n < 660) {
        ctx.setHorSit(sitInt, 660 - n);
    }
}

```

MarcacoesInvalidas (padrão)

```

/** Pares entrada/saída: lista vazia ou ímpar = inválida; data/hora de saída antes
da entrada = inválida. */
private boolean marcacoesInvalidas(List<MarcacaoRegra> mars) {
    if (mars == null || mars.isEmpty()) return true;
    if (mars.size() % 2 != 0) return true;
    for (int i = 0; i + 1 < mars.size(); i += 2) {
        // getData() tipicamente LocalDateTime — comparar o instante completo
        if (mars.get(i + 1).getData().isBefore(mars.get(i).getData())) {
            return true;
        }
    }
    return false;
}

```

Regras de negócio extras do cliente (tolerâncias, tipos de marcação) → `validacao_manual` / anexo — **sem** deixar o método vazio.

Outros pontos de customização (referência, não fluxo de menu)

Tipo	Base	APIs típicas
InicioCalculoColaborador	InicioCalculoColaborador	getMarcacoes, getTrocaHorario, incluirTrocaHorario, Periodo.getInstance

AposGravarApuracao	AposGravarApuracao	isTelaAcerto, getDiasCalculados, isUltimoColaborador
Manutenção BH	RegraManutencaoBH	getLancamentos() → ILancamentoBH
Fechamento BH	RegraFechamentoBH	getBancoHoras, getDataFinal, realizarFechamento
ContextoGeralRH	qualquer regra	getUsuarioAtivo, getUsuarioColaborador, getSaldoBancoHoras, getMesData

Checklist pré-entrega (operacional)

- ☐ `execute()` sem throws Exception
- ☐ `import com.senior.dataset.IEntity` (não g5...)
- ☐ Sem `.sc` em `R*`; `.sc` JSON com `{ ; id = filename`
- ☐ `MarcacaoRegra.getData().toLocalDate()`; `MarcacaoAnterior` só `diferencaMinutos`
- ☐ `TipCon` via SQL; IDs usuário `long`
- ☐ `Filename` = classe; auxiliares no nível da classe
- ☐ `ICursor` `close` no `finally`; `DBCcenter session.close`; `ContextSession` **sem** `close`

Itens pendentes / bloqueados até SDK (🚫)

Item	Status
Package/enum completo <code>TipoHoraExtra</code>	🚫 Bloqueado até SDK — usar só valores já citados no catálogo (<code>TipoIntervalo</code> etc.); demais → <code>validacao_manual</code> + perguntar enum do projeto. Não inventar como confirmada.
<code>@Transactional</code> vs container	🚫 Bloqueado até SDK — não anotar <code>@Transactional</code> sem evidência do projeto; deixar ao container.
<code>isNull/setNull</code> em <code>LocalDate</code> de <code>IEntity</code>	Template <code>IEntity</code> = <code>padrao_compilacao</code> ; se o SDK do projeto divergir → <code>validacao_manual</code> .
Capitalização getters <code>USU_*</code>	Norma fechada (ver Acesso a dados): espelhar nome do campo — fora desta tabela.

Sem `jar/docs/projeto` de referência, o teto de maturidade do treinamento fica **~9.x**; nota **10** exige fechar os 🚫 com evidência de SDK.

Não inventar itens 🚫 como `confirmada`.

Goldens ponta a ponta (sanitizados)

Referência obrigatória da Skill 1. Cada golden: LSP → inventário resumido → Java → checks Skill 5 esperados.

G-SIT — HorSit / minutos

LSP

```
Definir Numero nMin;  
nMin = HorSit[1];  
HorSit[1] = nMin + 60;
```

Inventário: HorSit[1] → getHorSit / setHorSit ; evidência confirmada ; contextoapuracao .

Java

```
@Rule(description = "G-SIT sanitizado")  
public class RegraGSIT extends Apuracao {  
    @Override  
    public void execute() {  
        ContextoApuracao ctx = getContainer().getContextoApuracao();  
        int nMin = ctx.getHorSit(1);  
        ctx.setHorSit(1, nMin + 60);  
    }  
}
```

Skill 5: PASS CHK-SITAPI , CHK-MIN , CHK-THROWS , CHK-STUB . FAIL se
getSituacao().setMinutos .

G-CUR — cursor R* (API / ContextSession, sem .sc)

LSP

```
Definir Alfa aNomEmp;  
CriarCursor('R030EMP');  
AbrirCursor('R030EMP');  
// leitura NomEmp ...  
FecharCursor('R030EMP');
```

Inventário: cursor R030EMP nativo → não .sc ; preferir API/readonly; fallback ContextSession; evidência
padrao_compilacao .

Java (padrão ContextSession se sem API semântica)

```
@Rule(description = "G-CUR sanitizado")  
public class RegraGCUR extends Apuracao {  
    @Override  
    public void execute() {  
        ContextoApuracao ctx = getContainer().getContextoApuracao();  
        String aNomEmp = "";  
        try {  
            ResultSet rs = ContextSession.getSession().executeQuery(  
                "SELECT NOMEMP FROM R030EMP WHERE NUMEMP = ?",  
                ctx.getColaborador().getNumEmp());  
            if (rs.next()) {  
                aNomEmp = rs.getString(0);  
            }  
        } catch (Exception ex) {
```

```

        ctx.mensagemLog("Erro G-CUR: " + ex.getMessage());
    }
    // ContextSession: não fechar sessão do container
}
}

```

Skill 5: PASS CHK-SCNAT (sem .sc em R*), CHK-SQL2 , CHK-DBSESS , CHK-STUB . Preferir API readonly/ getDefinicao... quando existir no catálogo.

G-USU — USU_* + I* + .sc

LSP

```

Definir Numero nCod;
CriarCursor('USU_TExemplo');
AbrirCursor('USU_TExemplo');
// lê USU_CodEmp ...
FecharCursor('USU_TExemplo');

```

Inventário: USU_TExemplo custom → IUsuTExemplo + ScUsuTExemplo.sc + ICursor; evidência padrao_compilacao .

Java (classe)

```

@Rule(description = "G-USU sanitizado")
public class RegraGUSU extends Apuracao {
    @Override
    public void execute() {
        ContextoApuracao ctx = getContainer().getContextoApuracao();
        ICursor<IUsuTExemplo> cur =
getContainer().getEntitySession().newCursor(IUsuTExemplo.class);
        try {
            cur.addFilter("USU_CodEmp = :e",
                new MappedParamProvider("e", ctx.getColaborador().getNumEmp()));
            cur.open();
            if (cur.first()) {
                IUsuTExemplo row = cur.read();
                int nCod = row.isUSU_CodEmpNull() ? 0 : row.getUSU_CodEmp();
                ctx.mensagemLog("USU_CodEmp=" + nCod);
            }
        } finally {
            cur.close();
        }
    }
}

```

Interface (trecho): getUSU_CodEmp / setUSU_CodEmp espelhando o campo.

.sc : JSON começando com { ; "id": "ScUsuTExemplo" = filename sem extensão; entity = package+interface; **somente** USU_* .

Skill 5: PASS CHK-FIN , CHK-SCJSON , CHK-SCID , CHK-SCNAT , CHK-STUB .

G-MIX — HorSit + cursor R* + USU_* (mesma regra)

LSP

```
Definir Numero nMin;  
Definir Alfa aNomEmp;  
Definir Numero nCod;  
nMin = HorSit[1];  
CriarCursor('R030EMP');  
AbrirCursor('R030EMP');  
// leitura NomEmp ...  
FecharCursor('R030EMP');  
CriarCursor('USU_TExemplo');  
AbrirCursor('USU_TExemplo');  
// lê USU_CodEmp ...  
FecharCursor('USU_TExemplo');  
HorSit[1] = nMin + 30;
```

Inventário (obrigatório separar):

Item	Destino	Evidência
HorSit[1]	getHorSit/setHorSit	confirmada
R030EMP	ContextSession/API — sem .sc	padrao_compilacao
USU_TExemplo	ICursor + IUUsuTExemplo + ScUsuTExemplo.sc	padrao_compilacao

Java (esqueleto sanitizado — uma classe; auxiliares no nível da classe)

```
@Rule(description = "G-MIX sanitizado")  
public class RegraGMIX extends Apuracao {  
    @Override  
    public void execute() {  
        ContextoApuracao ctx = getContainer().getContextoApuracao();  
        int nMin = ctx.getHorSit(1);  
        String aNomEmp = lerNomEmp(ctx);  
        int nCod = lerUsuCodEmp(ctx);  
        ctx.setHorSit(1, nMin + 30);  
        ctx.mensagemLog("emp=" + aNomEmp + " usuCod=" + nCod);  
    }  
  
    private String lerNomEmp(ContextoApuracao ctx) {  
        try {  
            ResultSet rs = ContextSession.getSession().executeQuery(  
                "SELECT NOMEMP FROM R030EMP WHERE NUMEMP = ?",  
                ctx.getColaborador().getNumEmp());  
            if (rs.next()) {  
                return rs.getString(0);  
            }  
        } catch (Exception ex) {  
            ctx.mensagemLog("Erro G-MIX R*: " + ex.getMessage());  
        }  
    }  
}
```



```

    }
    return "";
}

private int lerUsuCodEmp(ContextoApuracao ctx) {
    ICursor<IUsuTEmplo> cur =
getContainer().getEntitySession().newCursor(IUsuTEmplo.class);
    try {
        cur.addFilter("USU_CodEmp = :e",
            new MappedParamProvider("e", ctx.getColaborador().getNumEmp()));
        cur.open();
        if (cur.first()) {
            IUsuTEmplo row = cur.read();
            return row.isUSU_CodEmpNull() ? 0 : row.getUSU_CodEmp();
        }
    } finally {
        cur.close();
    }
    return 0;
}
}

```

Também emitir: IUsuTEmplo.java + ScUsuTEmplo.sc (só USU_*). **Nunca** .sc para R030EMP .

Skill 5: PASS CHK-SITAPI , CHK-SCNAT , CHK-FIN , CHK-SCID , CHK-STUB , CHK-NEST . FAIL se misturar .sc em R* ou getSituacao().setMinutos .

Relacionados

Núcleo Skill 3 · Skill 1 · Skill 2 · Skill 5